FABRIC / PRACTITIONER SERIES

SESSION 03 / PARTICIPANT LAB WORKBOOK

Build it. Explain it. Repeat it.

Hands-on low-code Fabric data engineering:
from source files to a governed reconciliation result.

October 28, 2026
Companion to the Session 03 teaching handbook

06	96	SQL-first
GUIDED LABS	MINUTES OF INTEGRATED PRACTICE	SYNTHETIC DATA / EXPLICIT CHECKS

SETUP / EXERCISES / EXPECTED RESULTS / RESET

YOUR PRACTICE PATH

One workload, six labs

Practice fits inside the existing three-hour session. Use the setup runbook first and retain evidence for every completion check.

00 **Setup, safety, assets, and reset** 03
Facilitator preflight before delivery

01 **Architecture decision framework** 09
8 minutes of integrated practice

02 **Repeatable ingestion** 12
18 minutes of integrated practice

03 **Curated SQL engineering** 15
25 minutes of integrated practice

04 **Consumption architecture** 18
20 minutes of integrated practice

05 **Operations and GCC High findings** 21
15 minutes of integrated practice

06 **Exit review and handoff** 24
10 minutes of integrated practice

Files and live execution

The companion source package contains the CSV, SQL, and DAX assets. Paths are relative to lessons/session_03/. Validate all service paths in the target GCC High tenant; offline checks do not establish tenant readiness.

FACILITATOR PREFLIGHT

Setup, safety, and reset

Audience: SQL, Power Query, and Power BI practitioners
Scope: Six labs supporting the existing Session 03 lessons, not additional sessions
Execution status: The package includes offline checks. Fabric, SharePoint, Power BI, and GCC High execution must be verified in the delivery tenant.

Start here

Use an approved, isolated training **Warehouse**, not a production Warehouse or the Lakehouse SQL analytics endpoint. Assign one Warehouse and one source folder per participant or pair. The scripts use fixed schema/object names, so simultaneous learners must not write to the same Warehouse.

These exercises fit inside the lesson allocations. They are not an additional 96 minutes after the three-hour session.

Lab	Hands-on allocation	Lesson allocation	Output
1. Architecture	8 minutes	25 minutes	Decision record and tool responsibilities
2. Ingestion	18 minutes	35 minutes	Verified replace-mode load and ingestion contract
3. SQL engineering	25 minutes	50 minutes	Baseline, rerun, correction, and guard-test evidence
4. Consumption	20 minutes	35 minutes	Shared model, measures, report, access/freshness decision
5. Operations	15 minutes	25 minutes	Runbook, finding, learning action, and ownership
6. Exit review	10 minutes	10 minutes	Completion record and accepted follow-ups

Facilitators should demonstrate the prepared SQL instead of having learners type every definition. Labs 1, 5, and 6 include decision/tabletop work; not every architecture or operating concept requires a new Fabric artifact.

Package inventory

All asset paths below are relative to `lessons/session_03/`.

Location	Contents
`labs/`	This setup/reset runbook and six participant guides
`artifacts/data/baseline/`	`system_a.csv` and `system_b.csv`
`artifacts/data/corrected/`	Replacement `system_b.csv`; changes only A002 from 250.00 to 200.00
`artifacts/data/expected/`	Baseline/corrected Gold business rows; blank CSV cells represent SQL nulls
`artifacts/sql/`	Setup, views, procedure, publication, checks, retry/guard exercises, and guarded reset/seed scripts
`artifacts/dax/`	Four measure definitions; create them individually
`artifacts/validate_lab_assets.py`	Offline fixture, source-alignment, and package-link checks
`artifacts/build_lab_workbook.py`	Rebuild the printable lab workbook using the existing handbook style
`pdf/session_03_lab_workbook.pdf`	Printable participant workbook including setup/reset

Do not upload the entire `artifacts/data/` tree into the ingestion folder. The dataflow must see only the two intended active files, not the corrected and expected-result copies.

Facilitator readiness gate

Complete this before class using the approved training identities. Write the actual identity role and evidence location, but never credentials.

Gate	Required evidence	If blocked
Workspace/ capacity	Correct tenant, capacity, item creation, and SQL execution	Use approved prepared evidence; record blocked paths
Source	Read access to the isolated SharePoint folder; matching synthetic files	SQL-only seed can support later labs, not prove ingestion
Destination	Dataflow can replace both existing text landing tables	Stop live ingestion and capture a scoped finding
SQL	Setup, procedure, baseline, checks, rerun, and guard test succeed	Use source/result walkthrough; do not claim live completion
Model/report	Verified creation route and actual mode; required refresh/framing	Use an approved alternative mode or label prepared evidence
Consumer	Intended read-only identity sees the approved report, not broad edit access	Access remains unverified; do not elevate to bypass it
Automation	Direct schedule and optional pipeline invocation tested separately	Explicitly supervise downstream steps
Safety	No concurrent writer; no production objects or real employee files	Do not proceed

If participant access is insufficient, use an approved facilitator demonstration and label it accordingly. Do not change production tenant settings or broad roles to rescue a lab.

Prepare the environment

1. Record the tenant, workspace, dedicated Warehouse, approved source folder, pair identifier, and technical owner.
2. Stop or disable **only the lab's** conflicting schedules with owner approval. Do not touch shared schedules.
3. Run `artifacts/sql/00_create_schemas.sql`, then `01_create_tables.sql`, in the dedicated Warehouse SQL editor. These create absent objects without clearing existing rows. If the names already contain unknown data or incompatible schemas, stop and use a fresh training Warehouse; the scripts do not repair existing objects.
4. Upload only `artifacts/data/baseline/system_a.csv` and `system_b.csv` to the approved active source folder.
5. Prepare Dataflow Gen2 connections and text-to-text destination mappings. Use Lab 2 for the detailed load instructions.
6. Preflight `02_create_normalized_view.sql`, `03_create_summary_view.sql`, and `04_create_refresh_procedure.sql`, each as a **separate complete execution**. `CREATE VIEW` and `CREATE PROCEDURE` must be the first statement in their batch. The files deliberately contain no `GO`; do not concatenate them.

7. The three `CREATE` definitions are one-time setup. If already installed by the facilitator, learners inspect them rather than rerunning `CREATE`. Record which objects already exist. If they need replacing, use a fresh Warehouse or an explicitly reviewed change, not an automatic drop.
8. Complete baseline ingestion. Confirm both refreshes and source scope before running `05_publish_baseline.sql`. Run `06_check_results.sql`, `07_test_rerun.sql`, and `08_test_blank_key_guard.sql` in separate executions. All should report `PASS` after the expected handled guard error.
9. Prepare the model/report route and a second, approved consumer identity. Refresh/framing and identity behavior depend on the actual mode.
10. Restore the baseline, retain source/refresh evidence, and agree an approved location for participant outputs.

The publication procedure's batch argument is only a label. The facilitator/operator must establish that both completed extracts belong together and prevent concurrent ingestion. This package does not add production-grade batch orchestration.

SQL script order and scope

Script	Purpose	When to run
`00_create_schemas.sql`	Create absent training schemas	Initial setup
`01_create_tables.sql`	Create absent Bronze/Gold tables	Initial setup
`02_create_normalized_view.sql`	Align names, types, and raw evidence	Once, after tables
`03_create_summary_view.sql`	Count rows/quality per source and key	Once, after normalized view
`04_create_refresh_procedure.sql`	Publish current Gold transactionally	Once, after views
`05_publish_baseline.sql`	Execute with the baseline batch label	After the intended baseline pair completes
`06_check_results.sql`	Assert row counts, label, keys, values, and classifications	Baseline by default; edit scenario for corrected data
`07_test_rerun.sql`	Compare business state before/after repeat execution	Verified, unchanged baseline only
`08_test_blank_key_guard.sql`	Reject a temporary blank key and preserve published data	Verified baseline, no other writers
`90_reset_rows.sql`	Clear only the three named training tables, preserving definitions	Optional explicit reset; confirmation required
`91_seed_baseline_sql_only.sql`	Replace training rows with the synthetic baseline	Approved SQL-only fallback; confirmation required

The guard test uses a session-local temporary snapshot and a transaction. The procedure rolls back the temporary source insertion; the test verifies the prior Gold rows, including publication metadata, are unchanged. This tests **pre-publication rejection**, not a simulated failure halfway through writing Gold. If any test reports an unexpected error, stop, inspect transaction state, and restore baseline before continuing.

Reset to the baseline between exercises

1. Confirm the dedicated Warehouse and stop competing lab writers.
2. Replace the active SharePoint `system_b.csv` with the baseline copy. Restore System A from its baseline copy if it was changed.
3. Run replace-mode ingestion for both files; verify successful completion and counts of six and five.
4. Run `05_publish_baseline.sql`, then `06_check_results.sql` with `@Scenario = 'baseline'`.

5. Perform the chosen model's freshness step and confirm unfiltered report totals of six records, one match, and five exceptions.
6. Record the reset and its evidence. Do not patch Gold rows directly.

If a full row reset is needed, edit the confirmation in `90_reset_rows.sql` to the exact phrase printed in that file, then execute it only after confirming the isolated target. It clears Bronze and Gold data, not schemas, views, the procedure, or external artifacts. Both sources must then be reloaded before publication.

For a SQL-only continuation when SharePoint/Dataflow is blocked, edit the confirmation in `91_seed_baseline_sql_only.sql` and execute it. It clears the current Gold result and seeds both Bronze tables. Follow with `05` and `06`. Mark ingestion as **blocked** or **prepared evidence reviewed**; never claim that a SQL insert validated Dataflow.

Completion states and evidence

For every lab, record one of: **participant completed**, **presenter demonstrated**, **prepared evidence reviewed**, or **blocked**. Keep screenshots/results in the approved delivery location with the lab number, pair identifier, timestamp, and input scenario. Sanitize connection errors and source locations before wider sharing.

Record live checks honestly. A package passing its offline validator does not prove Fabric SQL compatibility, SharePoint authentication, scheduled execution, Direct Lake availability, or consumer access.

From the training directory, run `python lessons/session_03/artifacts/validate_lab_assets.py` for the offline checks. Run `python lessons/session_03/artifacts/build_lab_workbook.py` to rebuild the workbook after editing its sources. The PDF builder reuses `lessons/build_pdfs.py` and its existing WeasyPrint, MarkdownIt, and Pygments dependencies.

Cleanup after the workshop

Agree retention with the artifact owner. Disable only approved lab schedules that should not remain active. Review dependencies before deleting a source folder, connection, model, report, or Warehouse through the service. No supplied script drops a Warehouse, schema, connection, or shared item. Preserve participant evidence and unresolved findings before any approved cleanup.

LAB 01 / 8 MINUTES

Architecture decision framework

> **WHAT YOU WILL PRODUCE**
>
> An architecture decision record, layer responsibilities, and a justified tool selection.

Before you begin

Read the setup and safety runbook. This is a paired design exercise; no service access is required. Refer to Lesson 1 for the teaching narrative.

Use the packaged synthetic files in `artifacts/data/baseline/`. System A has six rows, System B has five, and the combined sources contain six distinct normalized business keys.

Task 1: State the business question and grain

1. Open both CSVs in a text editor so spaces and duplicate rows remain visible.
2. Write the question the business needs answered without naming a Fabric product.
3. Define what one published row represents.
4. Identify which rows might create a misleading result if you joined the files without preparation.

Checkpoint: Your question compares records, amounts, and statuses. Your grain is one normalized record ID per comparison of the two complete snapshots. A005 is a duplicate-source ambiguity, not permission to publish extra business records.

Task 2: Choose the storage and transformation path

1. List the required operations: ingest the intended files, preserve evidence, normalize representations, evaluate quality, reconcile, and publish.
2. Compare a Warehouse/T-SQL path with a Lakehouse/notebook path.
3. Choose a starting point and explain how the team will support it.
4. Assign each layer a responsibility. Explain whether a view or a physical table is appropriate.
5. Identify why the Lakehouse SQL analytics endpoint is not a substitute for Warehouse T-SQL data writes.

Record the decision:

Field	Your answer
Business question and grain	Complete
Chosen starting point and required write operations	Complete
SQL/Power Query versus Spark skill fit	Complete
Bronze responsibility and evidence	Complete
Silver responsibility and object type	Complete
Gold responsibility and publication object	Complete
Rejected alternative and rationale	Complete
Assumptions requiring tenant validation	Complete
Revisit condition	Complete

Expected reasoning: Warehouse is a justified starting point for the relational, SQL-owned baseline. Both options use OneLake. Bronze preserves evidence; Silver normalizes and checks quality; Gold publishes business results. Three layers do not inherently require three workspaces.

Task 3: Match tools to work

Assign a responsibility or a reason not to use each tool:

Tool	Your decision
Dataflow Gen2	Complete
Pipeline	Complete
Copy job	Complete
Warehouse T-SQL	Complete
Notebook	Complete

Checkpoint: Dataflow shapes/loads; a pipeline coordinates dependencies; a copy job is considered for supported movement-oriented work; SQL performs relational curation. Notebooks require a justified capability need, not a desire to use every product.

Task 4: Revisit one assumption

The source now includes documents requiring custom Python extraction.

1. Identify the new operation.

2. State which part of the design might change and which parts can remain.
3. Name the capability and ownership assumptions you would verify before selecting a notebook/ Lakehouse path.

Completion check: Another pair can understand your selected option, rejected alternative, and revisit condition without asking what you meant.

Submit: The completed decision and tool tables with your completion state. No environment cleanup is needed.

LAB 02 / 18 MINUTES

Repeatable ingestion

> **WHAT YOU WILL PRODUCE**
>
> A verified replace-mode ingestion and a documented source, destination, and scheduling contract.

Before you begin

Follow the setup runbook. Required: an isolated Warehouse with the two Bronze tables, an approved SharePoint training folder containing only the active baseline files, and permission to author/run Dataflow Gen2.

Paths, connection names, and UI labels depend on the tenant. Use the approved connector and connection route; do not enter a commercial endpoint or create a gateway merely to bypass a Government connection failure.

Task 1: Inspect the source contract

1. Confirm the active files are `system_a.csv` and `system_b.csv`, copied from `artifacts/data/baseline/`.
2. Record the approved site/folder, comparison scope, source owner, and how the pair is confirmed complete.
3. Confirm that corrected/expected CSVs are not in the selected ingestion path.
4. Confirm that the connection's execution identity can read the files, not only your interactive browser identity.

Checkpoint: You can identify the exact pair and execution context before creating a query.

Task 2: Create or inspect the two queries

1. Open the approved training workspace and create or edit the prepared Dataflow Gen2.
2. Connect to the approved SharePoint source using the preflighted connection.
3. Filter the file list to the intended folder and exact System A file before reading its content.
4. Parse the CSV with a comma delimiter, promote the first row to headers, and inspect the three columns.
5. Remove or adjust automatic numeric conversions. Keep `RecordId`, `Amount`, and `Status` as text. Do not remove errors/duplicates to hide source problems.
6. Create the equivalent query for System B, retaining text `Reference`, `Total`, and `State`.
7. Inspect A005 twice in A and `not provided` for A006. Retain the spaces in B's first identifier until Silver normalization.

Checkpoint: System A preview has six rows; System B has five. Preview is not yet destination-load evidence.

Task 3: Configure and run destinations

1. Select the existing `bronze.system_a` table in the correct Warehouse for System A.
2. Map all three text columns explicitly.
3. Select `bronze.system_b` for System B and map its text columns.
4. Inspect the supported destination settings and select validated **replace** behavior for these complete snapshots. Do not assume settings for another destination apply.
5. Save/publish using the available experience and run the dataflow.
6. Wait for both destination loads to succeed; inspect refresh history and record the run evidence.
7. Execute this read-only query in the Warehouse:

```sql
SELECT 'A' AS source_system, COUNT(*) AS row_count
FROM bronze.system_a
UNION ALL
SELECT 'B', COUNT(*) FROM bronze.system_b;
```

Expect A = 6 and B = 5. Inspect the actual tables to confirm A005 remains duplicated and A006's raw amount remains `not provided`.

Task 4: Prove unchanged reload behavior

1. Run the unchanged ingestion again.
2. Wait for both loads and repeat the count/evidence checks.
3. Explain why the result is still six and five, rather than twelve and ten.
4. Explain why append alone would not correctly handle updates, deletes, or late arrivals in an incremental design.

Completion check: Two successful runs retain the same intended source state and evidence. If only one run was observed, record the rerun as unverified.

Task 5: Record the trigger and dependency boundary

1. Inspect the available direct scheduling configuration. Record its execution identity, time zone, owner, and intended time.
2. Enable a lab schedule only with owner approval. A configured schedule is not proof of a successful scheduled run; retain actual run evidence if already preflighted, or mark it pending.
3. If a pipeline is available and preflighted, inspect its Dataflow activity and downstream SQL success dependency. Do not start SQL on a separate estimated time delay.

4. If pipeline invocation is blocked, document direct ingestion scheduling plus supervised SQL/model steps.
5. Decide whether a copy job would satisfy these file-selection/Power Query requirements. Record source/destination availability as an assumption until verified.

Complete the contract:

Item	Your evidence or decision
Exact source pair and scope	Complete
Execution connection/owner and any approved gateway dependency	Complete
Destination names, column mappings, and update method	Complete
First and second run IDs/status/counts	Complete
Trigger and scheduled-execution evidence/state	Complete
What stops downstream work if either input fails	Complete
Full versus incremental choice and rationale	Complete

Failure branch: If one source fails, stop publication and record the finding. With facilitator approval, the SQL-only seed in the setup runbook can support later labs, but ingestion remains blocked or demonstrated, not participant-completed.

Submit: Contract and sanitized run/count evidence. Leave the complete baseline pair loaded; do not publish Gold yet.

LAB 03 / 25 MINUTES

Curated SQL engineering

> **WHAT YOU WILL PRODUCE**
>
> Explained reconciliation rows plus baseline, rerun, correction, and rejection evidence.

Before you begin

Use the setup runbook. Confirm both intended baseline snapshots are loaded (A = 6, B = 5), no ingestion is running, and the SQL editor targets your isolated **Warehouse**.

All SQL assets are under `artifacts/sql/`. Each file is a separate execution. If the facilitator already created the views/procedure, inspect those definitions instead of rerunning `CREATE`.

Task 1: Inspect Bronze and create or inspect Silver

1. Run `02_create_normalized_view.sql` if it is not already installed.
2. Query `silver.normalized_records`, selecting the raw and normalized fields.
3. Explain why B's `a001` becomes A001 and A006's amount becomes null instead of zero.
4. Run `03_create_summary_view.sql` if absent.
5. Query `silver.record_summary`, inspecting `source_row_count`, invalid counts, and projected amounts/statuses.
6. Verify A005 has two A rows and no arbitrarily chosen A amount. Verify A006 has one invalid A amount.
7. Query normalized rows with a null key. The baseline should return none.

Checkpoint: Duplicate, invalid-value, and blank-key handling are separate rules. A view does not freeze the underlying Bronze state.

Task 2: Publish and check the baseline

1. Run `04_create_refresh_procedure.sql` if absent; inspect its validation, transaction, replacement, and error path.
2. Confirm the operator's source-pair readiness check. The batch label does not establish readiness.
3. Run `05_publish_baseline.sql`.
4. Run `06_check_results.sql` with `@Scenario = 'baseline'`.
5. Save the `PASS`, classification counts, and six result rows.

Key	Expected classification
A001	MATCH
A002	AMOUNT_MISMATCH
A003	ONLY_IN_A
A004	ONLY_IN_B
A005	DUPLICATE_KEY
A006	INVALID_AMOUNT

The full expected business fields are in `artifacts/data/expected/baseline.csv`. Empty CSV cells represent SQL nulls.

Checkpoint: Six business keys, one match, five exceptions. Explain why a full outer join is required and why a direct duplicate-to-detail join would change the grain.

Task 3: Verify an unchanged rerun

1. Do not change or reload the sources while this test runs.
2. Run `07_test_rerun.sql`.
3. Record its result and repeat the baseline checker.
4. Explain why publication time may change while the intended business state does not.

Pass condition: Six rows remain, the business fields and batch label match the prior baseline, and the script reports `PASS`. This does not prove unattended orchestration.

Task 4: Follow a correction through the actual ingestion path

1. Predict the effect of changing B/A002 from 250.00 to 200.00.
2. Replace only the active SharePoint `system_b.csv` with `artifacts/data/corrected/system_b.csv`. Do not add a second file alongside it.
3. Reload both intended snapshots in replace mode and verify completion/counts. Leave System A unchanged.
4. Execute:

```sql
EXEC gold.usp_refresh_reconciliation
    @BatchId = 'training-2026-10-28-corrected';
```

1. In `06_check_results.sql`, change `@Scenario` to `'corrected'` and run it.
2. Compare the published fields with `artifacts/data/expected/corrected.csv`.

Pass condition: Six keys remain; A002 is now `MATCH`; totals are two matches and four exceptions. Other rows are unchanged.

If ingestion is blocked: Predict the outcome and review expected data. Label this task as prepared evidence reviewed, not an observed end-to-end correction.

Task 5: Restore baseline and test rejection

1. Follow the baseline reset sequence in the setup runbook: restore B's baseline file, reload both sources, run `05`, and run `06` with `'baseline'`.
2. With no other writers, run `08_test_blank_key_guard.sql` in one complete execution.
3. The script temporarily inserts a blank key in a transaction, expects error 50003, rolls back the insertion, and compares the prior Gold publication including its timestamp.
4. Record the final `PASS` and run the baseline checker again.

Pass condition: The blank row is absent, the previous Gold publication remains identical, and baseline checks pass. This verifies the pre-publication guard, not a simulated mid-write failure.

If an unexpected error appears, stop and contact the facilitator. Do not continue with a manually patched result or an open transaction.

Completion record

Check	State, evidence, and explanation
Raw-to-normalized trace for A001/A006	Complete
Duplicate handling and classification precedence	Complete
Baseline publication and exact result check	Complete
Unchanged rerun	Complete
Source correction and corrected check	Complete
Baseline restoration and blank-key rejection	Complete

Submit: Results and explanations. Leave the verified baseline published for Lab 4.

LAB 04 / 20 MINUTES

Consumption architecture

> **WHAT YOU WILL PRODUCE**
>
> A reusable model/report and a documented consumption, access, and freshness decision.

Before you begin

Use the setup runbook. Lab 3 must leave a verified baseline in `gold.reconciliation`. You need the approved semantic-model creation route, permission to author a training report, and an approved consumer test identity.

Direct Lake availability and creation experiences depend on the target tenant. Do not assume creating a Warehouse automatically creates the required model.

Task 1: Select and inspect the actual model mode

1. Open the approved model creation experience for the training Warehouse.
2. Select the physical `gold.reconciliation` table, not a report-specific copy of Bronze.
3. Create or inspect the explicitly named training semantic model. Name the model table `Reconciliation`.
4. Inspect the table's actual mode: Direct Lake, Import, or DirectQuery. Record it and the relevant connection/identity behavior.
5. If Direct Lake is used, identify the available variant and source path. Do not assume a SQL view supports the same behavior as a physical Delta table.
6. If the planned mode is unavailable, use an approved, preflighted alternative and record the deviation. A verified Import model is preferable to an unverified Direct Lake claim.

Checkpoint: You can distinguish model reuse from storage/query mode. Record-level reconciliation remains in Gold.

Task 2: Create the shared measures

1. Open `artifacts/dax/reconciliation_measures.dax`.
2. Create **each measure separately** in the model, in file order. The file is not a DAX query or a bulk-import script.
3. Confirm the model table name is exactly `Reconciliation`.
4. Format `Match Rate` as a percentage with two decimal places.
5. Explain why `Compared Records` counts six Gold keys, not eleven source rows.

Expected unfiltered baseline:

Measure	Expected value
Compared Records	6
Matched Records	1
Exception Records	5
Match Rate	16.67%

An exception is any classification other than `MATCH`, including quality exceptions.

Task 3: Build an investigable report

1. Create four cards for the measures.
2. Add a classification breakdown using classification and `Compared Records`.
3. Add a detail table with record ID, A/B amounts, A/B statuses, classification, batch ID, and publication time.
4. Trace A002's 200.00 versus 250.00 to the SQL result.
5. Trace A005 to its duplicate-source count; do not imply its missing projected A amount is zero.
6. Filter to `AMOUNT_MISMATCH`. Expect one compared record, zero matched, one exception, and a 0% match rate.
7. Clear filters and verify the baseline again. Explain how `KEEPFILTERS` preserves the selected classification context.

Checkpoint: Summary and detail agree with the comparison contract. A timestamp in a current-snapshot table does not create historical data.

Task 4: Compare direct SQL and test access

1. Query `gold.reconciliation` from the approved SQL editor/client and group by classification.
2. Compare those counts with the unfiltered report.
3. Decide which consumers actually need SQL access.
4. Open the approved report/app as the intended consumer, not as a workspace administrator. Have the facilitator perform this test if a separate approved identity is required.
5. Record whether viewing the report and authoring over the shared model require distinct permissions. Do not grant workspace editing solely to make a reader test pass.
6. Explain why semantic-model RLS and DAX measures do not automatically protect or define direct SQL/OneLake consumption.

If consumer testing is unavailable, mark it blocked. The presenter's successful view is not substitute evidence.

Task 5: Establish freshness

1. Record the Gold batch label and publication time.
2. Perform the mode-specific refresh/framing step or verify the applicable query/freshness behavior.
3. Compare report metadata and known totals with Gold.
4. Record the four gates: source completion, SQL publication, model freshness, and consumer visibility.

Complete the decision:

Item	Your answer and evidence
Audience, model name, source table	Complete
Actual mode/variant and rationale	Complete
Location of classifications versus reporting measures	Complete
Approved direct SQL consumers and permission owner	Complete
Consumer test identity role and result	Complete
Mode-specific freshness evidence	Complete
Model/report support owner	Complete

Submit: Model/report location, unfiltered and filtered results, access/freshness evidence, and decision. Do not attach credentials.

LAB 05 / 15 MINUTES

Operations and GCC High findings

> **WHAT YOU WILL PRODUCE**
>
> An ordered operating checklist, a scoped environment finding, and a separate learning action.

Before you begin

Use the setup runbook and your actual evidence from Labs 2-4. This lab combines monitoring inspection with a tabletop failure scenario. Do not break a shared connection, remove a real permission, or disrupt production to demonstrate failure.

Task 1: Inspect one known run

1. Open the completed Dataflow refresh history and identify the intended source pair.
2. Record successful destination counts and the source-completion gate.
3. Inspect the SQL check result, batch label, and publication time.
4. Inspect model/report freshness evidence using the actual mode.
5. Identify the last successful publication and where an operator finds it after a later failure.

Checkpoint: Technical activity status, business checks, and consumer evidence are three different kinds of proof. If only prepared evidence is available, label it.

Task 2: Write the ordered runbook

For each gate, write an executable instruction or a specific supervised check, not just a check mark.

Order	Gate	Actual instruction/ evidence	Owner and failure response
1	Source pair ready	Complete	Complete
2	Both ingestion outputs successful	Complete	Complete
3	Source alignment, counts, and quality checked	Complete	Complete
4	SQL publication and result checks pass	Complete	Complete
5	Model's freshness requirement met	Complete	Complete
6	Intended consumer sees expected publication	Complete	Complete
7	Run outcome and findings recorded	Complete	Complete

If a pipeline was verified, record the actual activities and success dependencies. If not, name the person supervising SQL/model steps. Separate schedules with estimated time gaps are not successful dependency orchestration.

Task 3: Respond to a failed source

Use this scenario:

> System A finished today. System B failed. Yesterday's report still opens and looks reasonable.

1. State which work must stop and what published result remains valid.
2. State what users should be told about freshness.
3. Identify evidence required before retrying.
4. Explain why a SQL transaction protecting Gold does not repair partially refreshed Bronze tables.
5. Explain how the response differs if Gold succeeded but only model refresh failed.

Expected response: Do not compare mismatched snapshots. Preserve and identify the last successful Gold/report state, record the failure and owner, and resume only after a matching complete pair is verified. If only the model stage failed, investigate/retry that stage rather than automatically reloading everything.

Task 4: Separate three issue types

Classify these observations:

1. A participant cannot explain why a duplicate key is an exception.
2. A source contains A006's `not provided` amount.

3. A pipeline invocation using the approved connection returns an authentication error.

Expected categories: learning need, source-data exception, environment finding. The third observation does not establish a Government-wide unsupported capability.

Record one learning action:

Skill	Evidence	Focused next practice	Owner
Complete	Complete	Complete	Agree

Record an environment finding, using a real observed failure or clearly labeling the tabletop case as **simulated**:

Field	Your entry
Observation and investigation status	Complete
Tenant/workspace/item scope and tested role	Complete; no secrets
Timestamp and sanitized run/error evidence	Complete or mark simulated
Reproduction, expected behavior, actual behavior	Complete
Blocked stage and operating impact	Complete
Owner and next diagnostic/support action	Agree
Approved interim approach, manual effort, and review condition	Complete

Task 5: Assign support and change responsibilities

1. Name the source, connection, ingestion, SQL/data, model/report, and support owners, with backup coverage.
2. Confirm the minimum appropriate workspace/artifact permissions instead of assigning broad edit roles to readers.
3. Apply existing naming conventions. Explain why `test` in a name does not provide isolation.
4. Describe review, known-result checks, promotion approval, and recovery if the business changes invalid amounts from exceptions to publication blockers.
5. Record owner-approved schedule retention and artifact cleanup. Do not delete shared items as a lab exercise.

Completion check: A second operator can follow your gate sequence, identify the failure boundary, and find an accountable owner without the original presenter.

Submit: Runbook, evidence, separate issue records, and agreed owners.

LAB 06 / 10 MINUTES

Exit review and handoff

> **WHAT YOU WILL PRODUCE**
>
> An evidence-based completion record, accepted follow-up actions, and participant track alignment.

Before you begin

Bring the outputs from Labs 1-5 and the setup/reset runbook. This is a review and teach-back, not another implementation exercise.

Task 1: Verify the handoff package

1. Exchange evidence with another pair or the facilitator.
2. For each output below, record one state: **participant completed**, **presenter demonstrated**, **prepared evidence reviewed**, or **blocked**.
3. Identify any missing prerequisite or hidden step that would prevent someone else from repeating the work.

Output	State	Evidence location	Remaining gap
Architecture record and tool choices	Complete	Complete	Complete
Ingestion contract and observed rerun	Complete	Complete	Complete
SQL baseline/correction/rerun/guard checks	Complete	Complete	Complete
Model, measures, report, and actual mode	Complete	Complete	Complete
Consumer access and freshness	Complete	Complete	Complete
Operating checklist and separate logs	Complete	Complete	Complete

Do not upgrade a reviewed screenshot to participant completion or a manual step to verified unattended automation.

Task 2: Teach back the design

Without reading SQL aloud, explain:

1. Why Warehouse fits the baseline and what would justify a Lakehouse/notebook component.
2. Why landing text and raw values matter.

3. Why A005 is not a match and A006 is not zero.
4. Why reloading and rerunning unchanged input must not duplicate the intended state.
5. Why scheduled ingestion alone does not prove complete orchestration.
6. How the consumer can establish the publication's identity and freshness.

Acceptance check: The listener can trace source -> landing -> quality -> Gold -> model -> report and identify the gates, not just name the components.

Task 3: Convert gaps into actions

Choose the most important unresolved items and agree:

Item and current state	Accepted owner	Next action	Completion evidence	Agreed review point
Complete	Agree	Complete	Complete	Agree
Complete	Agree	Complete	Complete	Agree

Use specific actions such as "Retest pipeline invocation with the intended connection and capture a sanitized run result," rather than "Fix pipelines." Do not invent ownership or dates without agreement.

Expected source-data exceptions are not automatically implementation defects. Keep them with the source/data owner.

Task 4: Select the next learning track

Record each participant's intended responsibility, preferred primary track, and one skill to develop. Confirm with the participant and technical lead.

Responsibility	Primary track	Next skill
Complete	SQL/Warehouse or notebook/Spark	Complete

SQL/Warehouse emphasizes relational modeling, procedures, loads, and quality/operations. Notebook/Spark emphasizes Python/PySpark, Delta processing, parameters, and engineering orchestration. Neither is a maturity ranking, and today does not require everyone to learn Python.

Final completion check

The package is ready for a learning handoff when another practitioner can find the source files, follow the approved setup sequence, reproduce the expected results, and identify the next action when a stage is blocked.

Tenant checks still pending must remain visible. Completion of the learning exercises is not production approval.

Submit: Accepted completion record, evidence locations, actions, and track choices. Use the setup runbook for baseline restoration and approved cleanup.